SLIIT
Discover Your Future

IT1170 – Data Structures and Algorithms

Lecture 10

Divide and Conquer - 2

Merge sort

Merge Sort is a sorting algorithm based on divide and conquer.

Its worst-case running time has a lower order of growth than insertion sort.

The merge sort algorithm closely follows the divide-and-conquer paradigm. Intuitively, it operates as follows.

- **Divide:** Divide the *n*-element sequence to be sorted into two subsequences of *n*/2 elements each.
- **Conquer:** Sort the two subsequences recursively using merge sort.
- **Combine:** Merge the two sorted subsequences to produce the sorted answer.

Merge sort

Divide by splitting into two subarrays $A[p \,.\,.\, q]$ and $A[q + 1 \,.\,.\, r]$, where q is the halfway point of $A[p \,.\,.\, r]$.

Conquer by recursively sorting the two subarrays $A[p \,.\,.\, q]$ and $A[q + 1 \,.\,.\, r]$.

Combine by merging the two sorted subarrays $A[p \,.\,.\, q]$ and $A[q + 1 \,.\,.\, r]$ to produce a single sorted subarray $A[p \,.\,.\, r]$.

To accomplish this step, we'll define a procedure MERGE*(A, p, q, r)*.

Merge sort procedure

Input : A an array in the range 1 to n.
Output : Sorted array A.

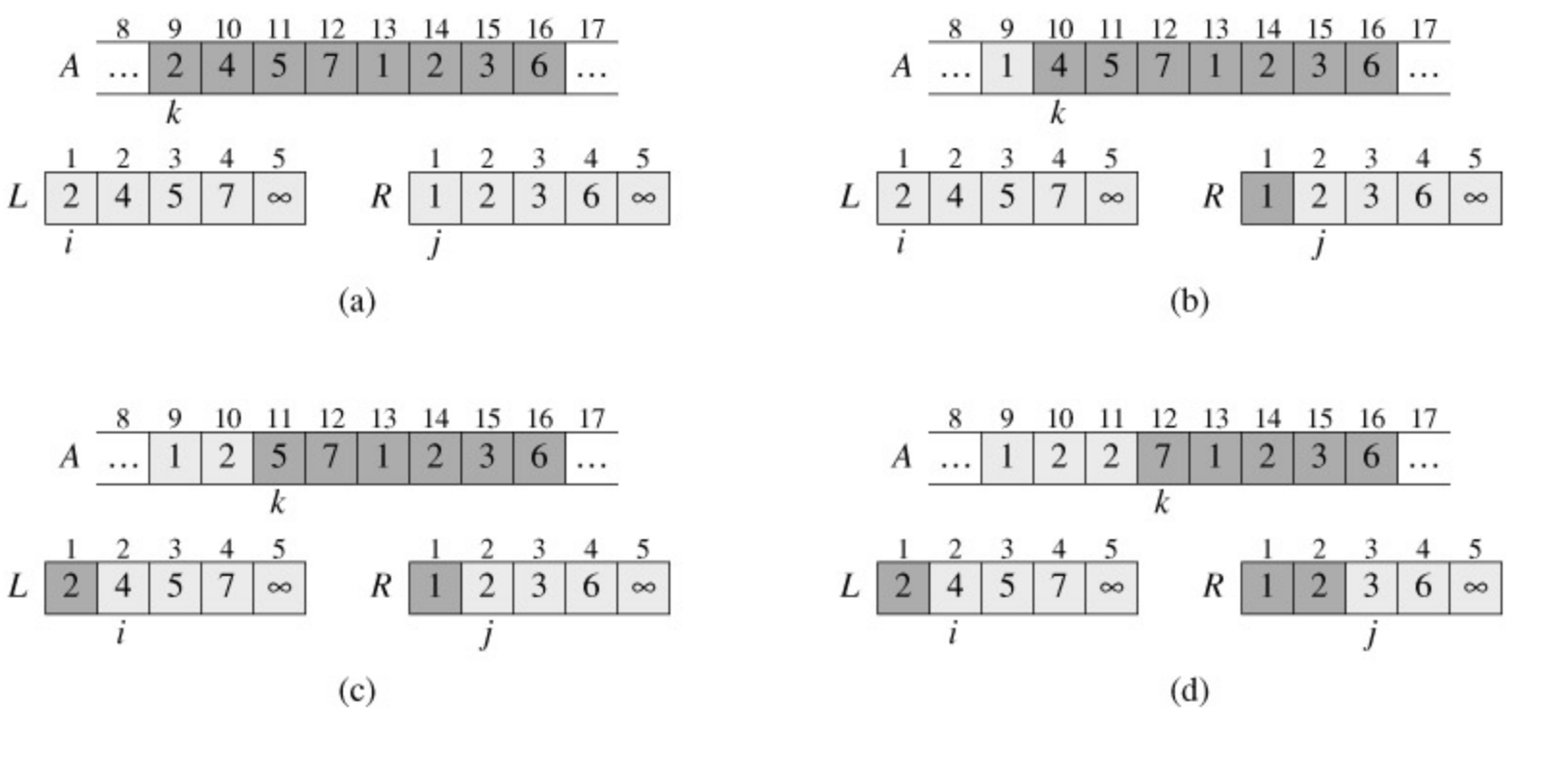
MERGESORT (A, p, r)

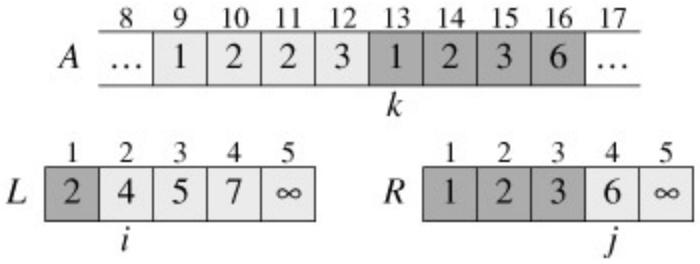
1. **if** $p < r$
2. $q = \lfloor (p+r)/2 \rfloor$
3. **MERGESORT** (A, p, q)
4. **MERGESORT** (A, q+1, r)
5. **MERGE** (A, p, q, r)

Merge procedure

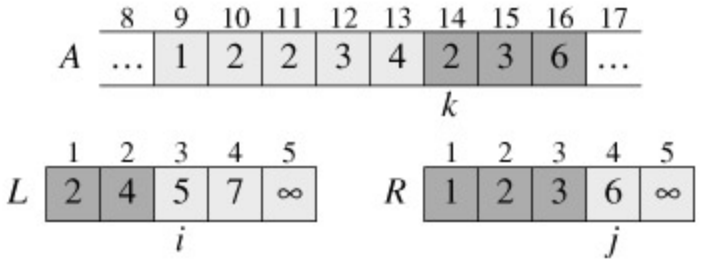
```
MERGE(A, p, q, r)
    n1 = q - p + 1
    n2 = r - q
    create arrays L[1.. n1 + 1] and R[1.. n2 + 1]
    for i = 1 to n1
        L[i] = A[p + i - 1]
    for j = 1 to n2
        R[j] = A[q + j]
    L[n1 + 1] = ∞
    R[n2 + 1] = ∞
    i = 1
    j = 1
    for k = p to r
        if L[i] ≤ R[j]
            A[k] = L[i]
            i = i + 1
        else A[k] = R[j]
            j = j + 1
```

Illustration when the subarray $A[9..16]$ contains the sequence ⟨2, 4, 5, 7, 1, 2, 3, 6⟩

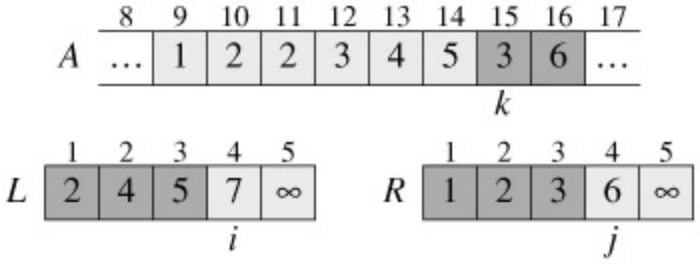



8 9 10 11 12 13 14 15 16 17
A … 1 2 2 3 1 2 3 6 …
k
1 2 3 4 5
L 2 4 5 7 ∞
i
1 2 3 4 5
R 1 2 3 6 ∞
j

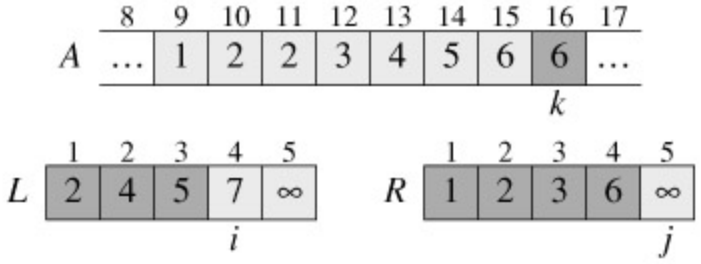
(e)


8 9 10 11 12 13 14 15 16 17
A … 1 2 2 3 4 2 3 6 …
k
1 2 3 4 5
L 2 4 5 7 ∞
i
1 2 3 4 5
R 1 2 3 6 ∞
j

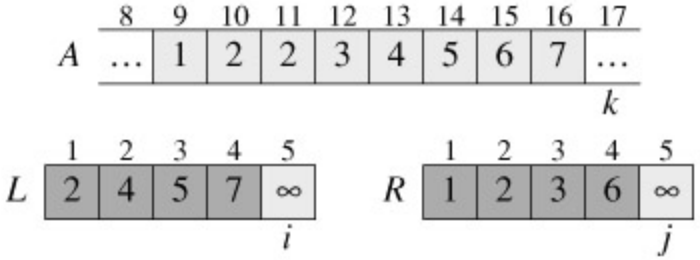
(f)


8 9 10 11 12 13 14 15 16 17
A … 1 2 2 3 4 5 3 6 …
k
1 2 3 4 5
L 2 4 5 7 ∞
i
1 2 3 4 5
R 1 2 3 6 ∞
j

(g)


8 9 10 11 12 13 14 15 16 17
A … 1 2 2 3 4 5 6 6 …
k
1 2 3 4 5
L 2 4 5 7 ∞
i
1 2 3 4 5
R 1 2 3 6 ∞
j

(h)


8 9 10 11 12 13 14 15 16 17
A … 1 2 2 3 4 5 6 7 …
k
1 2 3 4 5
L 2 4 5 7 ∞
i
1 2 3 4 5
R 1 2 3 6 ∞
j

(i)

Analysis of Merge Sort

- To find the middle of the sub array will take $\Theta(1)$.
- To recursively solve each sub problem will take $2T(n/2)$.
- To combine sub arrays will take $\Theta(n)$.

Therefore $\mathbf{T(n)=2T(n/2)+\Theta(n)+\Theta(1)}$

We can ignore $\Theta(1)$ term.

$$\mathbf{T(n)=2T(n/2)+\Theta(n)}$$

```
MERGESORT (A, p, r)
1.  if p < r
2.      q = ⌊(p+r )/2⌋
3.      MERGESORT (A, p, q)
4.      MERGESORT (A, q+1, r)
5.      MERGE (A, p, q, r)
```

Analysis of Merge Sort

$T(n) = 2T(n/2) + cn$

$2T(n/2) = 4T(n/4) + 2cn/2$

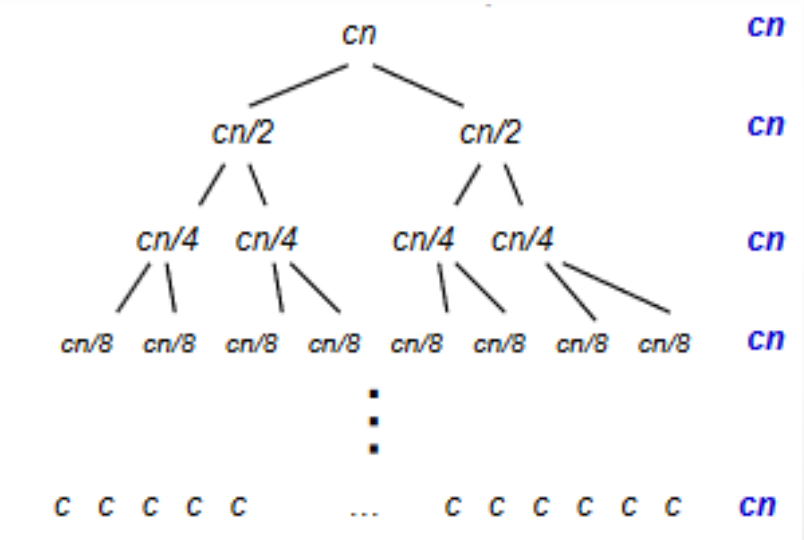
$4T(n/4) = 8T(n/8) + 4cn/4$

-

-

-

$T(2) = nT(1) + (n/2)\, c*2$



---

add and cancel:

$$T(n) = nT(1) + cn + cn + \ldots + cn$$

$$= nT(1) + cn * \log_2 n = \Theta(n\log n)$$